

CE2007

Microprocessor-based System Design

ARM Cortex-M Programming

Topics Covered

- Part 1: Cortex-M Assembly Language Programming
- Part 2: Cortex-M Embedded C Programming

Objective: Provide sufficient background information for student to perform laboratory exercises

Cortex-M

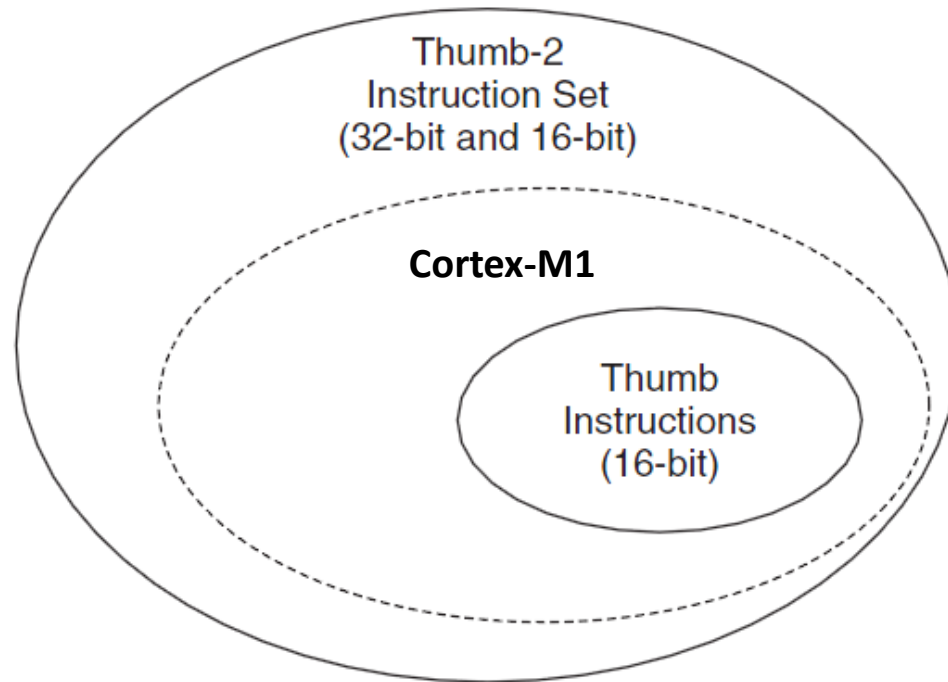
Assembly Language Programming

Background of A32, Thumb and Thumb-2 Instruction Set

- ARM processors were originally (1985) designed to run with fixed length 32-bit instructions (A32 ISA)
 - Need 32-bit wide memory to store instructions
 - Not cost effective as memory devices were expensive in the early days
- A restricted 16-bit instruction set was hence introduced in 1994, known as the Thumb ISA
 - Less memory required due to smaller code size, hence lower cost and lower operating power
 - Need more instructions to execute certain functions that can be supported more efficiently in A32 ISA
- **Thumb-2** (T32) ISA is hence developed in 2003
 - Consist of the original Thumb instruction as well as new 32-bit instructions which is selected based on the best features of A32 ISA
 - Instruction set consists of mix 16-bit/32-bit instructions (variable-length instruction set)

Cortex-M1 Instruction Set

- Cortex-M1 is based on the ARMv6-M architecture
 - Includes all base 16-bit Thumb instructions and a few Thumb-2 32-bit instructions
 - Enables writing very tight and efficient processor code, which is ideal for the limited memory found in deeply embedded applications



Data Types Supported by Cortex-M

- 32-bit Words
 - Address location is aligned to four byte boundaries, hence the lowest two address bits are zero (xxxxxx00b)
 - Data are stored as Little Endian (LSB at lower address) by default
- 16-bit Half-Words
 - Address location is aligned to two byte boundaries, hence the lowest address bit is zero (xxxxxxx0b)
- 8-bit Bytes
 - Can be placed on any byte boundary address

- Instruction format has four fields separated by spaces or tabs
 - **Label** (Optional) starts in first column and is used to identify the position memory of the instruction
 - **Opcode** specifies the instruction to execute
 - **Operand** specifies where to find the data to execute the instruction (can have up to four operands in T32)
 - **Comment** (optional), separated by comma, is used to describe the code making it easier to understand

Examples:

```
START      MOV R1, #100      ; sets R1=100
LOOP      ADD R1, R1, #10    ; R1=R1+10
           B    LOOP          ; Branch to LOOP
```

- There are three basic classes of instructions:
 - Data Transfer
 - Memory-Register Load/Store
 - Data Operation
 - Register movement
 - Arithmetic/Logical
 - Comparison and test
 - Flow Control
 - Branch
 - Conditional Execution

Note: We will only cover the basic instructions that is sufficient for Lab 1 exercises

Load-Store Architecture

- ARM is a Load-Store architecture
 - Can only operate on data already stored in registers
- So if you want to add the content of two memory address, M1 and M2, and store in address M3, the ARM would have to:
 - **Load** content of M1 into a register, R1
 - **Load** content of M2 into a register, R2
 - Perform an **addition** ADD R3, R1, R2
 - **Store** content of register R3 into address M3
- Two types of instructions are involved in the example above:
 - Data transfer (Load and Store) - Slow
 - Data operation (ADD, AND, EOR, SUB, etc.) - Fast

Question 1

Why is it challenging to write assembly programs using the 16-bit Thumb instruction set for the Load-Store Architecture?

(To be discussed in lectures)

Data Transfer Instructions

- ARM only provides two basic instructions for data transfer between memory and processor registers

- Load and Store

Examples :

LDR R0, [R1]

- load R0 with the content of the memory location pointed to in R1
- R1 is known as the base address register

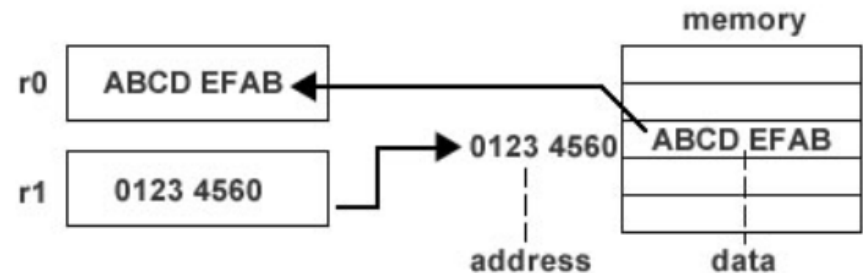
STR R2, [R1, #offset]

- store content of R2 to memory location with address pointed to in R1+offset

LDRB R0, [R1] ; load byte sized data

STRH R0, [R1] ; store halfword sized data

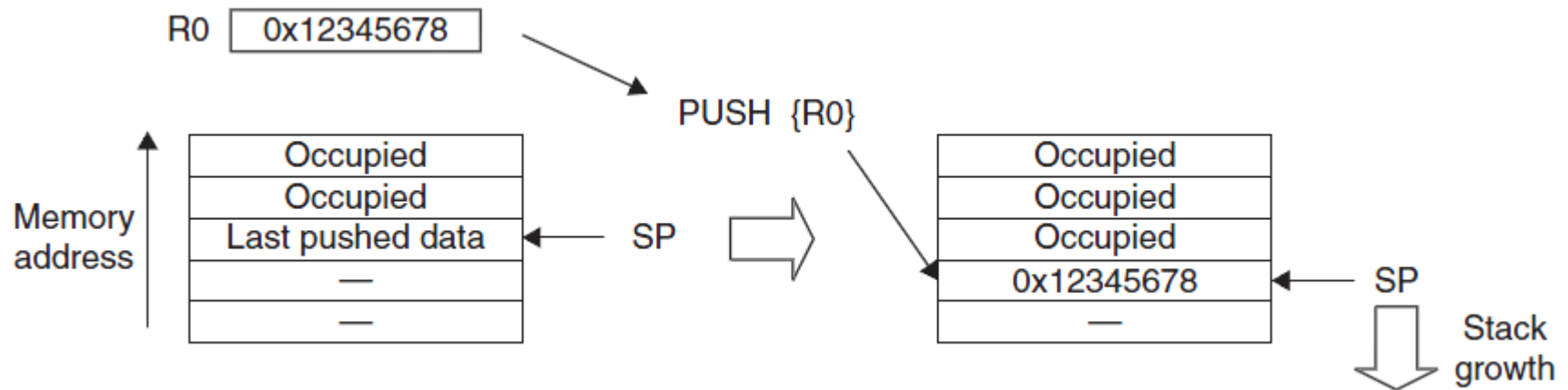
LDRSB R0, [R1] ; load signed byte



Stack Operation (PUSH)

- Stack is used to save to memory, the content of register(s) that may be affected when performing a subroutine call

PUSH {R0} ; R13=R13-4, then Memory[R13] = R0



Example:

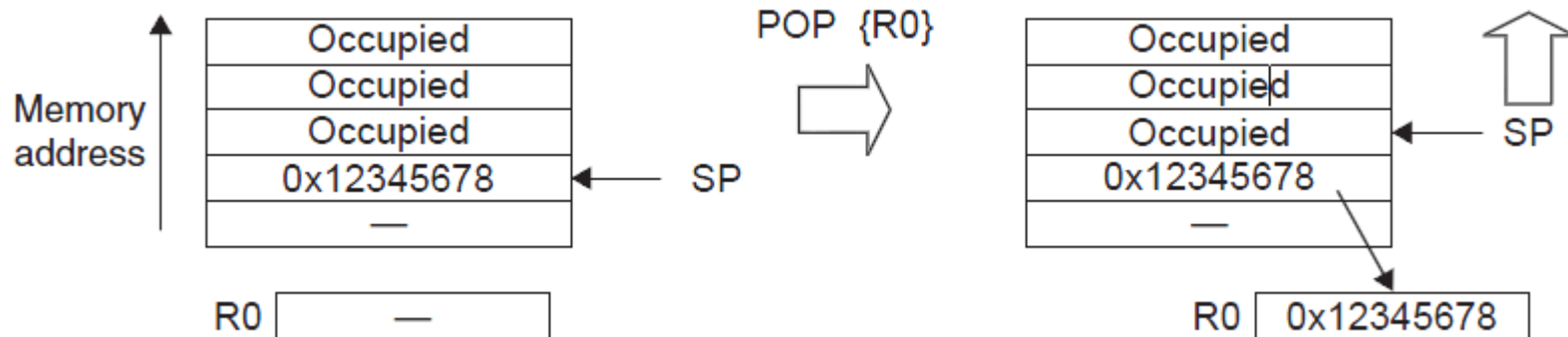
To save multiple-register content on a stack

PUSH {R0, R4-R8, LR}

- this instruction 'pushes' the registers content to the stack

Stack Operation (POP)

POP {R0} ; R0 = Memory[R13], then R13=R13+4



The corresponding instruction to 'pop' the content from the stack is

POP {R0, R4-R8, PC}

Question 2

The figure below shows the initial contents of a memory segment, registers R0, R1 and R2 of the Cortex-M1 processor. How would the contents of the memory, registers R0, R1 and R2 change after the following instructions are executed consecutively?

- LDR R0, [R1]
- STR R2, [R0]

Memory

1234 2000	2468 1234	R0	1234 2004
1234 2004	1234 2008	R1	1234 200C
1234 2008	F01F DACA	R2	245C ABAB
1234 200C	1234 2000		

↑ Address ↑ Data

A

Memory

1234 2000	245C ABAB	R0	1234 200C
1234 2004	1234 2008	R1	1234 200C
1234 2008	F01F DACA	R2	245C ABAB
1234 200C	245C ABAB		

↑ Address ↑ Data

C

Memory

1234 2000	245C ABAB	R0	1234 2000
1234 2004	1234 2008	R1	1234 200C
1234 2008	F01F DACA	R2	245C ABAB
1234 200C	1234 2000		

↑ Address ↑ Data

B

Memory

1234 2000	2468 1234	R0	1234 2000
1234 2004	245C ABAB	R1	1234 200C
1234 2008	F01F DACA	R2	245C ABAB
1234 200C	1234 2000		

↑ Address ↑ Data

D

Memory

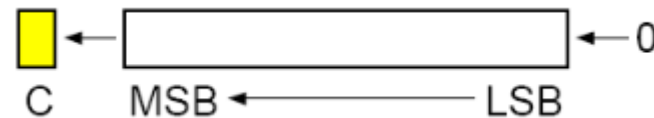
1234 2000	2468 1234	R0	1234 2004
1234 2004	245C ABAB	R1	1234 200C
1234 2008	F01F DACA	R2	245C ABAB
1234 200C	1234 2004		

↑ Address ↑ Data

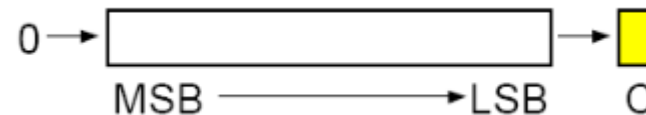
Bit Shifting Operations

- Bit shifting increases the power and flexibility of many data processing instructions

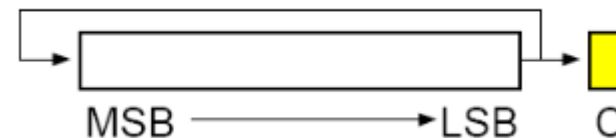
LSL Logical Shift Left



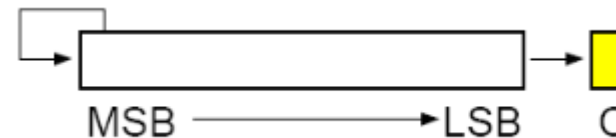
LSR Logical Shift Right



ROR Rotate Right



ASR Arithmetic Shift Right



Example:

LSL R1 ,R1 ,#2 ; left shift R1 by 2-bit

Register Move Instruction

- MOV instructions simply move the second operand to the destination register Rd

Examples:

- Copy from one register to another register

MOV R1 , R2 ; R1 = R2

- Copy an immediate value to a register

MOV R2 , #0x32 ; R2 = 0x32

A special form of the LDR instruction is also available to load the register with immediate value:

LDR R0 , =511 ; R0= 511

MOV Variations

- Usage of bit-shifting operation with MOV
 - Performs very fast multiplication and division
 - Left shift = multiply by 2
 - Right shift = divide by 2

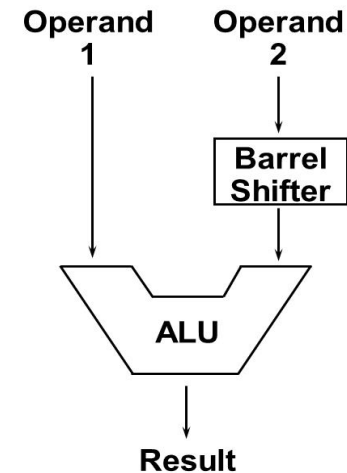
Example:

```
MOV    R1, R2, LSL #2      ; R1=R2<<2
                        ; i.e. R1=4xR2
```

- Convenient way to perform bit masking

Example:

```
MOV    R1, #0x1, LSL 15   ; R1=0x00000001<<15
                        ;   = 0x00008000
                        ; i.e. set bit 15
```



Logical Instructions

Logical instructions perform boolean operations on a pair of operands

- AND Logical bitwise
- ORR Logical bitwise OR
- EOR Logical exclusive OR
- BIC Logical bit clear

AND Truth Table

A	B	Z = A . B
0	0 *	0
0	1	0
1	0 *	0
1	1	1

Useful for bit masking purposes. (e.g., clear status bit or change interrupt masks)

Examples:

```
AND    R0,R1,#0xff    ; R0=lowest byte of R1
```

```
BIC    R1,R1,#0x10    ; clear bit 5 of R1
```

Arithmetic Instructions

- Perform addition and subtraction of 32-bit signed and unsigned values
 - **ADD** and **SUB** Addition and Subtraction
 - **ADC** and **SUBC** Addition and Subtraction including Carry

Example:

```
SUB    R0 , R1 , R2    ; R0 = R1 – R2
```

Conditional flag can also be specified, e.g. used for loop counting purposes:

```
SUBS   R0 , R0 , #1    ; R0 = R0–1, S suffix to set  
                        ; Zero flag when R0 = zero
```

- Flow control instructions are used to divert the flow of the program
 - Used to implement loops and subroutine calls

The most basic branch instruction is

B: Branch

Example:

```
        B    SKIP ; branch to SKIP
        :
SKIP    MOV  R1,R2
```

- Branch becomes more flexible when it is used together with the conditional codes

Conditional Branch

- A conditional branch is constructed by adding a conditional suffix to the basic branch instruction
 - The branch will take effect if the condition is met

Examples:

BEQ	Branch on equal
BNE	Branch on not equal
BGE	Branch when greater or equal to (signed)
BLT	Branch when less than (signed)

- To use these instructions, they have to be preceded with the appropriate data processing instructions that set the respective status flags

Example:

```
SUBS    R0, R1, #5    ; R0=R1-5
BEQ     SKIP          ; if Z=1, branch
```

Branch and Link

- The instruction Branch and Link (**BL**)
 - Is used for executing a subroutine or a function call, where the program has to return to the code after the branch instruction
- The link register (LR, which is R14) saves the address of the next instruction after **BL** before executing the branch

**BL label ; Branch to a labeled address and
save return address in LR**

- When the function completes its execution
 - It loads its PC with the LR register value to effect the return

Branch Instructions

- Branch and Exchange (**BX**)
 - Provides a mechanism for program control to return to calling process when the argument is LR

Example:

```
      :  
      BL  SUBR      ; branch to subroutine, LR holds  
                   ; address of next instruction  
      MOV R1,R4     ; get value returned in R4 into R1  
      :  
SUBR      ; subroutine starts here  
      :  
      MOV R4,#rslt  ; store return result in r4  
      BX  LR        ; load PC with LR, which cause  
                   ; program to return
```


Assembler Directives

- Used to assist and control the assembling of the assembly program

Examples:

```
AREA  ARMex, CODE, READONLY
                                ; Name this block of code ARMex
ENTRY                                ; Mark first instruction to execute
START MOV    R0,#10 ; Set up parameters
:
END
```

- The AREA directive marks the start of a section
 - Names the section and sets its attributes that are separated by commas
 - Example above defines a read-only section called ARMex that contains code

Assembler Directives (Cont.)

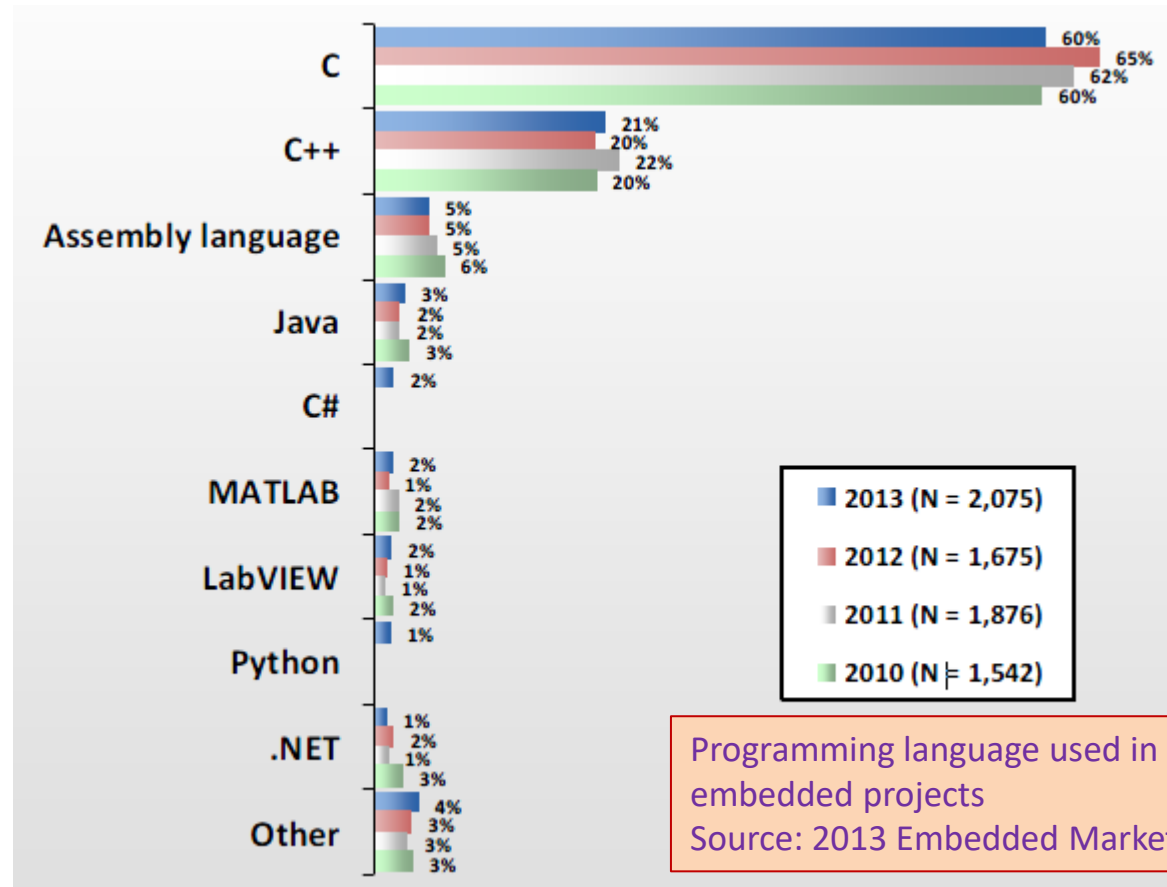
- The **ENTRY** directive declares an entry point to a program
 - The application code begins executing at the label **START**
- The **END** directive instructs the assembler to stop processing this source file
 - Every assembly language source module must finish with and **END** directive on a line by itself
 - Any lines following the **END** directive are ignored by the assembler
- Other directives can be used to add variable and constants
 - **DCD** Place a 32-bit word into the memory
 - **ALIGN** Align next data to word boundary
 - **SPACE** Reserve byte-size space in memory
 - **EQU** Give a symbolic name to a numeric constant

Cortex-M

Embedded C Programming

C for Embedded Systems

- C language is particularly suitable for embedded applications
 - With the exception of Apps for Android which is based on Java



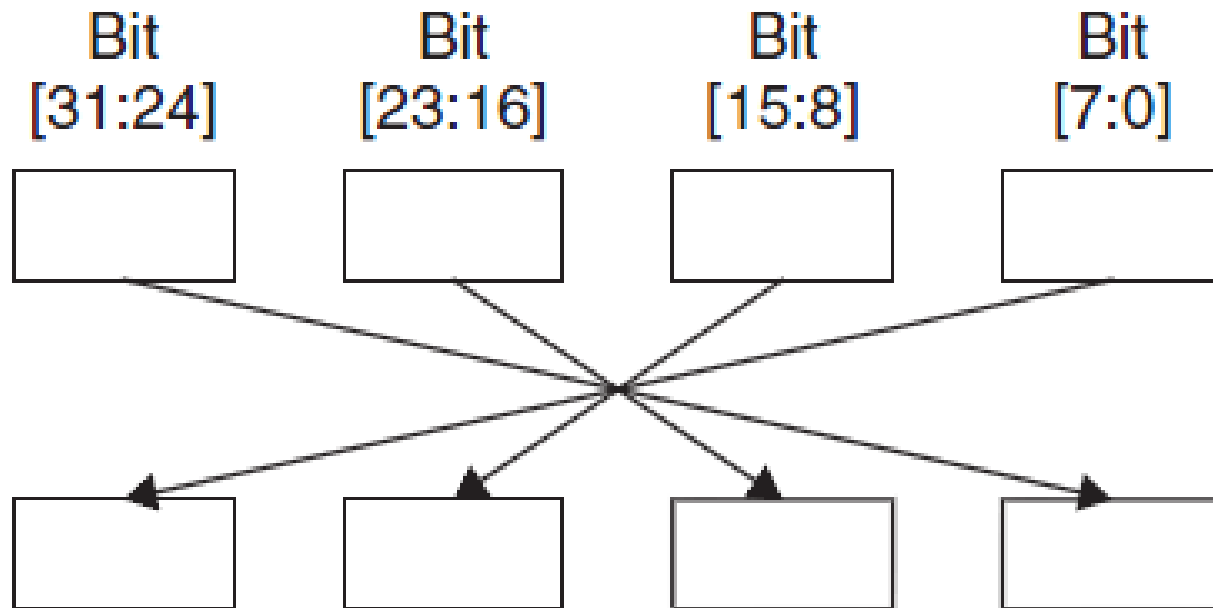
C for Embedded Systems

- C provides the various benefits of programming in high level languages and supports useful features, such as
 - Bit-wise data manipulation (common for controlling I/O devices)
 - Direct low level access to hardware platform (important for initialization of processor and devices)
 - Portable across wide variety of processor families and different platforms
- Disadvantages when compared to assembly language
 - Limited/no direct access to core register and stack
 - No direct control over instruction sequence generation
 - No direct control over stack usage

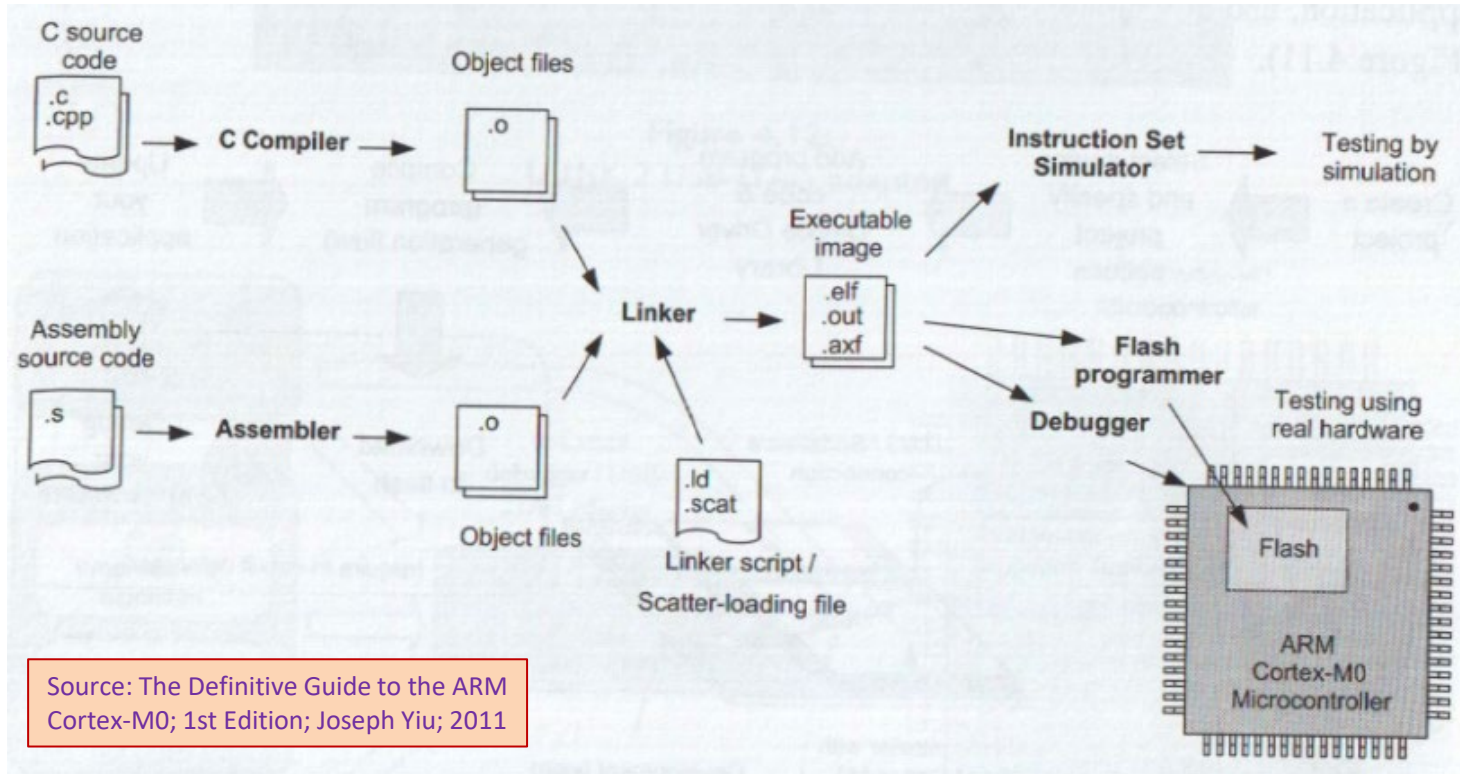
Programming in Assembly Language

- Allows direct access to instructions that cannot be generated with C

Example: Byte-Reverse Word



Mix C and Assembly Code

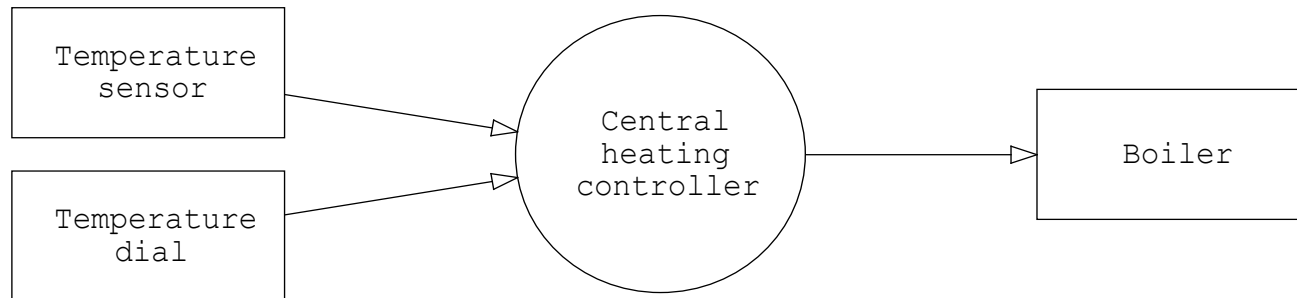


- **ARM Architecture Procedure Call Standard (AAPCS)** has specified the following for interfaces between functions in C programming
 - R0, R1, R2 and R3 are to be used to pass input parameters between functions in a C program
 - R0 is to be used to store return results
 - R12 is used as a scratchpad register

Hosted vs. Free-Standing Environment

- Normal C based programs typically deal with 'data processing' types of applications
 - Mainly involve file I/O and numeric and string oriented data
 - Standard library functions (e.g. `stdio.h`) are heavily used for these types of applications, typically running with OS support
 - Known as Hosted Environment
- But embedded C applications need to meet different requirements
 - Device I/O bit manipulation, real-time process control
 - Common to be stand-alone programs without OS support
 - Known as Free-Standing Environment

Typical Structure of Embedded C Programs



```
void main(void)
{
    /* Init the system */
    C_HEAT_Init();

    while(1) /* 'for ever' (Super Loop) */
    {
        /* Find out what temperature the user requires
           (via the user interface) */
        C_HEAT_Get_Required_Temperature();

        /* Find out what the current room temperature is
           (via temperature sensor) */
        C_HEAT_Get_Actual_Temperature();

        /* Adjust the gas burner, as required */
        C_HEAT_Control_Boiler();
    }
}
```


- C program always start with the `main()` function
 - But how does the processor gets initialized before the `main()` is called?
- In Hosted Environment (such as Personal Computer)
 - Processor is initialized by BIOS or OS
- In Free Standing Environment
 - A separate *start-up code* is needed/provided
 - The start-up code is created automatically in IDE such as Keil μ Vision, pre-configured for the chosen target device
- This start-up code will be combined with the C code during the linking process to generate the final executable code

Start-Up Code (Cont.)

- The start-up code
 - Is typically written in assembly language
 - Performs low level initializations of the processor (e.g. timer, interrupt mechanism)
 - Setup the proper environment needed for the execution of the C program (e.g. stack and heap allocation)
 - Follows/requires certain naming convention

Example of Start-Up code

```
Stack_Size      EQU      0x00000200
                 AREA     STACK, NOINIT, READWRITE, ALIGN=3
Stack_Mem        SPACE    Stack_Size
__initial_sp

Heap_Size        EQU      0x00000400
                 AREA     HEAP, NOINIT, READWRITE, ALIGN=3
__heap_base
Heap_Mem         SPACE    Heap_Size
__heap_limit

; Vector Table Mapped to Address 0 at Reset
                 AREA     RESET, DATA, READONLY
                 EXPORT   __Vectors
__Vectors        DCD      __initial_sp          ; Top of Stack
                 DCD      Reset_Handler         ; Reset Handler
                 DCD      NMI_Handler           ; NMI Handler
                 DCD      HardFault_Handler     ; Hard Fault Handler
                 :

Reset_Handler    PROC
                 EXPORT   Reset_Handler
                 IMPORT   __main
                 LDR      R0, =__main
                 BX       R0
                 ENDP
```

Question 1

Analyse the Reset_Handler in the Start-Up code to understand how the main function is executed upon reset.

(To be discussed in lectures)

- In embedded application, data is usually of integer types (e.g. register of an I/O device)
 - But it is often critical to know the exact bit size of the integer data to be manipulated
 - Especially when accessing registers and I/O devices
- Example: What would the bit size for variables declared as `char`, `int`, `short`, `long`?
 - Depends on the target platform
 - These are provided in the ISO C89, which is more suitable for conventional C applications running in hosted environment
- For embedded applications
 - ISO C99 standard removes ambiguities

- Integer data type available under ISO C99

Data type	Size	Range
<code>int8_t</code>	8 bits	-128 to +127
<code>int16_t</code>	16 bits	-32768 to +32767
<code>int32_t</code>	32 bits	-? to +?
<code>int64_t</code>	64 bits	-? to +?
<code>uint8_t</code>	8 bits	0 to 255
<code>uint16_t</code>	16 bits	0 to 65535
<code>uint32_t</code>	32 bits	0 to +?
<code>uint64_t</code>	64 bits	0 to +?

Access of Memory Mapped I/O

- With I/O devices mapped into the memory address space
 - Accessing their data in C is most conveniently achieved by using pointer identifier

Example: The following defines pointer identifiers for I/O devices found on the DE0 board mapped at their respective addresses

```
#define DE0_LEDS          *(volatile uint32_t *) (0xA0000000))  
#define DE0_SLIDE_SW     *(volatile uint32_t *) (0xA0001000))  
#define DE0_7SED_DISP    *(volatile uint32_t *) (0xA0004000))
```

Access of the device can then be simple done using statements such as

```
DE0_LEDS=0x3FF;           // turn on all LEDs  
DE0_LEDS=DE0_SLIDE_SW     // what would this achieve?
```

'volatile'

- The **volatile** keyword used in the declaration of identifier or variable plays an important role for I/O devices in embedded C programming
 - It prevents compiler optimization of the I/O related code, which involve re-arranging the code to improve the code size/speed performance but may affect the correctness of the I/O operation during execution

Example: The following code may not work without the **volatile** declaration when compiler optimization is enabled during compilation

```
While(1) {DE0_LEDs = DE0_SLIDE_SW}
```


C Bitwise Operations

a) To check/test the value of a single bit

```
if ((DE0_SLIDE_SW & 0x10) !=0 ) // check bit 5
```

or simply

```
if (DE0_SLIDE_SW & 1<<5) // check bit 5
```

A	B	Z = A . B
0	0 *	0
0	1	0
1	0 *	0
1	1	1

b) To set the value of a single bit

```
DE0_LEDS = DE0_LEDS | 1<<3; // this is to ?
```

or more elegantly (but not as clear)

```
DE0_LEDS |= 1<<3;
```

A	B	Z = A+B
0	0	0
0	1 *	1
1	0	1
1	1 *	1

c) To clear the value of a single bit

```
DE0_LEDS &= ~(1<<5);
```

d) To toggle the value of a single bit

```
DE0_LEDS ^= (1<<5);
```

A	B	Z = A⊕B
0	0	0
0	1 *	1
1	0	1
1	1 *	0

Question 2

- Using bitwise operations ('&', '^', '|', '~'), which of the following C statement will toggle bits **7** and **3** of an 8-bit register ST_REG?
- A. ST_REG &= 0x90;
- B. ST_REG ^= 0x88;
- C. ST_REG |= 0x64;
- D. ST_REG &= ~(0x90);